Токенизация текста

Виктор Китов
victorkitov.github.io

```
Large Language Models (LLMs), such as GPT-3 and GPT-4, util
ize a process called tokenization. Tokenization involves br
eaking down text into smaller units, known as tokens, which
the model can process and understand. These tokens can rang
e from individual characters to entire words or even larger
chunks, depending on the model. For GPT-3 and GPT-4, a Byte
Pair Encoding (BPE) tokenizer is used. BPE is a subword tok
enization technique that allows the model to dynamically bu
ild a vocabulary during training, efficiently representing
common words and word fragments. Although the core tokeniza
tion process remains similar across different versions of t
hese models, the specific implementation can vary based on
the model's architecture and training objectives.
```

Содержание

1 Виды токенизации

2 Токенизаторы BPE и byte level BPE

3 Токенизатор WordPiece

Токенизация

- При обработке текста, его нужно токенизировать - представить последовательностью минимальных смысловых единиц (токенами):

$$\text{text} \to t_1 t_2 ... t_N$$

- Каждый токен заменяется его эмбеддингом $t_i \to \mathbf{e}_i \in \mathbb{R}^D$.
- Последовательность эмбеддингов уже подаётся в модель:

$$f(\mathbf{e}_1 \mathbf{e}_1 ... \mathbf{e}_N) = \widehat{y}$$

- Сами эмбеддинги могут быть
  - фиксированными (one-hot)
  - настраиваться с параметрами модели
  - браться из другой предобученной модели (transfer learning)
- Какие виды токенизации вы знаете?

Простейшие виды токенизации

- Пословные токенизаторы:

$$\text{Playing games.} \rightarrow \text{[Playing] [games] [.]}$$

- Посимвольные токенизаторы:

$$\text{Playing games.} \rightarrow \text{[P] [l] [a] [y] [i] [g] [ ] [g] [a] [m] [e] [s] [.]}$$

- Какие преимущества и недостатки?

Простейшие виды токенизации

- Пословные токенизаторы:

$$\text{Playing games.} \rightarrow \text{[Playing] [games] [.]}$$

- Посимвольные токенизаторы:

$$\text{Playing games.} \rightarrow \text{[P] [l] [a] [y] [i] [g] [ ] [g] [a] [m] [e] [s] [.]}$$

- Какие преимущества и недостатки?
- Посимвольная:
  - малый словарь
  - нет проблемы новых слов (out-of-vocabulary, OOV)
- Пословная:
  - компактнее представление текста
  - быстрая обработка
  - модели легче устанавливать зависимости

Современный подход

- Современные токенизаторы делят текст на часто встречающиеся подслова.
- Пример:

$$\text{Playing games.} \rightarrow \text{[Play] [ing] [ ] [game] [s] [.]}$$

- $\downarrow$ размер словаря, особенно для морфологически богатых языков (как русский, немецкий, финский, турецкий)
- **Быстрая обработка**, т.к. токены крупные.
- **Обработка новых слов**, которых не было в обучении.
  - модель поймёт новое слово playing, если ранее видела play, work, working.
- Генерируя текст токен за токеном, **модель может генерировать новые слова**!

Современные токенизаторы

- **BPE (Byte-Pair Encoding)**[1]
  - стартует от отдельных символов
  - объединяет фрагменты, чаще всего встречающиеся вместе
  - используется в GPT, RoBERTa, XLM-R.
- **Byte-level BPE** - версия BPE, стартующая от отдельных байтов
  - полностью покрывает $\forall$ текст (+эмодзи, редкие символы)
  - используется в GPT-2, GPT-3, RoBERTa.
- **WordPiece**[2]
  - стартует от отдельных символов
  - строит словарь, максимизируя вероятность корпуса
  - объединяет неслучайно часто совстречающиеся фрагменты.
  - используется в BERT, ALBERT, DistilBERT.

[1] Sennrich R., Haddow B., Birch A. Neural machine translation of rare words with subword units.

[2] Schuster M., Nakajima K. Japanese and korean voice search.

Нормализация

Нормализация текста - важный этап перед обработкой:

- заглавные буквы $\rightarrow$ строчные
  - apple, Apple, APPLE $\rightarrow$ apple
- удаление акцентов
  - ё, é $\rightarrow$ e: ёлка$\rightarrow$елка, café$\rightarrow$cafe
- замена табуляций и переносов строк пробелами
- замена повторных пробелов одним
- удаление невидимых системных символов
  - например, символ конца файла

Нормализация

Нормализация текста - важный этап перед обработкой:

- заглавные буквы $\rightarrow$ строчные
  - apple, Apple, APPLE $\rightarrow$ apple
- удаление акцентов
  - ë, é $\rightarrow$ e: ёлка$\rightarrow$елка, café$\rightarrow$cafe
- замена табуляций и переносов строк пробелами
- замена повторных пробелов одним
- удаление невидимых системных символов
  - например, символ конца файла

Некоторые из шагов могут сознательно пропускаться.

- можем сохранять заглавные буквы
  (чтобы лучше понимать эмоции)
- можем сохранять акценты
  (чтобы лучше понимать авторский стиль)

Содержание

1 Виды токенизации

2 Токенизаторы BPE и byte level BPE

3 Токенизатор WordPiece

Идея алгоритма

- **Начальные токены**:
  - символы языка
  - знаки препинания
  - спец. токены ([PAD], [BOS], [EOS]...)
- **Обучение**:
  - В алгоритме итеративно объединяются соседние токены, имеющие максимальную частоту совстречаемости.
  - Порядок выученных слияний фиксируется.
- **Применение**:
  - выученные правила слияний применяются последовательно в то же порядке

Алгоритм обучения

1. **Подготовка:** Текст нормализуется, делится на слова, в конце каждого слова добавляется маркер конца слова ##.
2. **Инициализация:** Словарь заполняется
   - уникальными символами корпуса;
   - знаками препинания и спец. токенами.
3. **Итерация:**
   - Считаем #(A,B) для всех соседних фрагментов A,B.
   - Находим пару с максимальной частотой.
   - Добавляем правило слияния A B $\rightarrow$ AB и добавляем AB в словарь.
   - Обновляем корпус, заменяя все вхождения A B на AB .
4. **Остановка:** достигнут целевой размер словаря.

Различение границ слов за счёт

- Алгоритм использует маркер ## в конце слова, чтобы отличать финальные фрагменты от серединных.
- Пример:
  - pay ing bill $\rightarrow$ payingbill
  - pay ing## bill $\rightarrow$ paying bill
- Это обеспечивает восстановление пробелов.

Обучение: инициализация

Обучающий корпус: *pay paying lay.*

Инициализация (разбиение на токены):

1. p, a, y, ##
2. p, a, y, i, n, g, ##
3. l, a, y, ##

- Общее кол-во токенов N = 4 + 7 + 4 = 15.
- Начальный словарь символов: {p , l , a , y , i , n , g , # #}.

Обучение: пример подбора слияний

Обучающий корпус: *pay paying lay*.

Рассмотрим пару (a, y):

- Данная пара встречается во всех 3-х словах корпуса.
- #(a,y)=3 . Это максимальное значение на первом шаге.
- **Слияние 1:** a,y$\rightarrow$ay .

Рассмотрим пару (p, ay) после первого слияния:

- Она встречается в словах *pay* и *paying*.
- #(p,ay)=2 .
- **Слияние 2:** p , ay $\rightarrow$ pay.

Результат: Постепенно формируются токены ay, pay, lay.

Применение

Обучающий корпус: *pay paying lay.*

После обучения получим упорядоченный список правил:

1. $\mathrm{a\ y} \rightarrow \mathrm{ay}$
2. $\mathrm{ay\ \#\#} \rightarrow \mathrm{ay\#\#}$
3. $\mathrm{p\ ay} \rightarrow \mathrm{pay}$
4. $\mathrm{l\ ay} \rightarrow \mathrm{lay}$
5. ...

Применение:

1. Слово разбивается на символы.
2. Все выученные правила слияния применяются к нему строго в порядке их изучения при обучении.

Применение: пример

Обучающий корпус: *pay paying lay.*

- **lay** $\rightarrow$

Применение: пример

Обучающий корпус: *pay paying lay.*

- **lay** $\rightarrow$ [lay##]
  - последовательно применились правила:
    (a, y) $\rightarrow$ ay, (l, ay) $\rightarrow$ lay, (lay, ##)$\rightarrow$lay## .
- **laying** $\rightarrow$

Применение: пример

Обучающий корпус: *pay paying lay.*

- **lay** $\rightarrow$ **[lay##]**
  - последовательно применились правила:
    (a, y) $\rightarrow$ ay, (l, ay) $\rightarrow$ lay, (lay, ##)$\rightarrow$lay## .
- **laying** $\rightarrow$ **[lay] [ing##]**
  - Склеились lay и суффикс $\mathrm{ing}\#\#$.
    - Правило $\mathrm{lay}\#\#$ не сработало, так как далее шел символ i.
- **layed** $\rightarrow$

Применение: пример

Обучающий корпус: *pay paying lay.*

- **lay** $\rightarrow$ **[lay##]**
  - последовательно применились правила:
    (a, y) $\rightarrow$ ay, (l, ay) $\rightarrow$ lay, (lay, ##)$\rightarrow$lay## .
- **laying** $\rightarrow$ **[lay] [ing##]**
  - Склеились lay и суффикс $\mathrm{ing}\#\#$.
    - Правило $\mathrm{lay}\#\#$ не сработало, так как далее шел символ i.
- **layed** $\rightarrow$ **[lay] [UNK] [UNK] [##]**
  - Корень lay распознан успешно.
  - Символов e, d не было в обучении.

Byte-level BPE (BBPE)

- **Проблема:** BPE на уровне Unicode может встретить неизвестные символы (редкие иероглифы, эмодзи).
- **Решение (GPT-2, Llama):** Byte-level BPE, использующий в качестве базового словаря 256 байтов вместо символов Unicode.
- **Механизм:**
  - Любой текст кодируется в UTF-8.
  - Любой символ представляется последовательностью от 1 до 4 байтов.
  - BPE объединяет наиболее частые байтовые пары.
- **Преимущества:**
  - Никаких [UNK], $\forall$ последовательность байтов может быть токенизирована.
  - В базовом словаре всего 256 токенов вместо десятков тысяч Unicode-символов.
  - Модель может выучить зависимости даже внутри мультиязычных или зашумленных данных.

Анализ BPE и byte-level BPE

Преимущества:

- Эффективное сжатие данных (частые слова — один токен).
- Отсутствие OOV при использовании byte-level BPE.

Недостатки:

- Необходимо выполнять много слияний при инференсе.
- Редкие, но важные связи могут игнорироваться (зависимость от частотности).

Используется в моделях **GPT-2/3/4, RoBERTa, Llama,** ...

Содержание

1 Виды токенизации

2 Токенизаторы BPE и byte level BPE

3 Токенизатор WordPiece

WordPiece - максимизация правдоподобия

- Пусть корпус C состоит из слов $\{w_1, \ldots, w_n\}$.
- Каждое w_i разбивается на токены $t_{i,1}, \ldots, t_{i,K_i}$,
  Вероятность слова-произведение вероятностей его токенов:

$$P(w_i) = \prod_{j=1}^{K_i} P(t_{i,j}) \implies L = \prod_{tokens \in C} P(t)$$

WordPiece - максимизация правдоподобия

- Пусть корпус C состоит из слов $\{w_1, \dots, w_n\}$.
- Каждое w_i разбивается на токены $t_{i,1}, \dots, t_{i,K_i}$,
  Вероятность слова-произведение вероятностей его токенов:

$$P(w_i) = \prod_{j=1}^{K_i} P(t_{i,j}) \implies L = \prod_{tokens \in C} P(t)$$

- Подберём такое правило слияний, чтобы ↑правдоподобие:

$$L = P(C) = \prod_{i=1}^{N} P(w_i) \to \max_{tokens}$$

WordPiece - максимизация правдоподобия

- Пусть корпус C состоит из слов $\{w_1, \dots, w_n\}$.
- Каждое w_i разбивается на токены $t_{i,1}, \dots, t_{i,K_i}$,
  Вероятность слова-произведение вероятностей его токенов:

$$P(w_i) = \prod_{j=1}^{K_i} P(t_{i,j}) \implies L = \prod_{tokens \in C} P(t)$$

- Подберём такое правило слияний, чтобы ↑правдоподобие:

$$L = P(C) = \prod_{i=1}^{N} P(w_i) \to \max_{tokens}$$

- Объединение пары (A, B) в токен AB меняет L.
  Если $\#(A, B) = N_{AB}$:

$$Gain = \frac{L_{new}}{L_{old}} = \left(\frac{P(AB)}{P(A) \cdot P(B)} \right)^{N_{AB}}$$

Идея алгоритма

- В алгоритме итеративно объединяются соседние фрагменты, объединение которых приводит к максимальному gain.
- Упрощение[3]: степень N_{AB} часто опускается в реализации, чтобы находить статистически более сильные связи.
  - не давая преимущества исключительно самым частым

$$Gain \approx \frac{P(AB)}{P(A) \cdot P(B)}$$

- Для устойчивости часто вводят доп. ограничение:

(A, B) объединяется, если $\#(A, B) \geq$ threshold.

[3] HuggingFace LLM course.

Алгоритм обучения

1. **Подготовка:** Текст нормализуется и делится на слова.
2. **Инициализация:** Словарь V заполняется
   - символами языка
   - знаками препинания
   - спец. токенами [CLS], [SEP], [PAD] и т.д.
3. **Итерация:**
   - Считаем частоты токенов $P(t) = \frac{\#(t)}{\#\text{Tokens}}$.
   - Считаем $Score = \frac{P(AB)}{P(A)P(B)}$ для всех соседних пар в словах.
   - Объединяем пару с Max Score в один токен.
     - объединяются только символы и группы символов
     - знаки препинания и спец. токены не участвуют
4. **Остановка:** Когда достигнут целевой размер словаря.
   - возможна досрочная остановка, если max score<threshold

Обратимость токенизации за счёт

- Алгоритм различает токены в начале и середине слова
  - используя префикс ##
- Пример: play (начало слова), ##ing (середина)
- Без префиксов последовательность [play, ing] может означать:
  - playing (одно слово)
  - play ing (два отдельных слова)
- С префиксом ## декодирование однозначно:
  - [play, ##ing] $\rightarrow$ playing
  - [play, ing] $\rightarrow$ play ing

Обучение: инициализация

Обучающий корпус: *playing replayed replaying*.

Инициализация (разбиение на токены):

1. p, ##l, ##a, ##y, ##i, ##n, ##g
2. r, ##e, ##p, ##l, ##a, ##y, ##e, ##d
3. r, ##e, ##p, ##l, ##a, ##y, ##i, ##n, ##g

Общее кол-во токенов $N = 7 + 8 + 9 = 24$.

Обучение: пример вычислений

Обучающий корпус: *playing replayed replaying*.

Рассмотрим пару (p, ##l):

- $P(p) = 1/24, P(\#\#l) = 3/24, P(p, \#\#l) = 1/24$
- $Score\,(p, \#\#l) = \frac{1/24}{(1/24 \cdot 3/24)} = \frac{1}{3/24} = \mathbf{8}$

Рассмотрим пару (##i, ##n):

- $Score = \frac{2/24}{(2/24 \cdot 2/24)} = \frac{1}{2/24} = \mathbf{12}$

Результат: (##i, ##n) выигрывает и объединяется в ##in.

Итоговый словарь и применение

Обучающий корпус: *playing replayed replaying*.

После ряда итераций в словаре появятся:

- play (начало слова)
- ##play (в середине слова)
- ##ing (суффикс)
- re (приставка)
- ##ed (суффикс)
- ...

При применении для каждого слова

1. сопоставляется самый длинный префикс (быстро!)
2. потом самый длинный префикс для остатка
3. и т.д.

Если слово невозможно полностью разбить на токены словаря, всё слово→[UNK].

Применение: пример

Обучающий корпус: *playing replayed replaying*.

Рассмотрим применение для слов *play*, *played*, *player*.

1. **play** $\rightarrow$

Применение: пример

Обучающий корпус: *playing replayed replaying*.

Рассмотрим применение для слов *play*, *played*, *player*.

1. **play** $\rightarrow$ **[play]**
   - есть токен целиком
2. **played** $\rightarrow$

Применение: пример

Обучающий корпус: *playing replayed replaying*.

Рассмотрим применение для слов *play, played, player*.

1. **play → [play]**
   - есть токен целиком
2. **played → [play, ##ed]**
   - Самый длинный префикс: play, потом $\mathrm{ed} \to \#\#\mathrm{ed}$
   - Остатокищем Найдено.
   - **Токены:**
3. **player → [UNK]**
   - Префикс: play. Остаток er.
   - $\#\#\mathrm{er}$ не найден, но найден $\#\#\mathrm{e}$, остаток $\#\#\mathrm{r}$.
   - $\#\#\mathrm{r}$ не найден (есть только r), поэтому всё слово помечается [UNK].

Анализ метода

Достоинства:

- Быстрий инференс - пытаемся начать с самого длинного префикса.
- Математическая база ($\uparrow$ правдоподобия).
- Хорошо выделяет морфологию (корни, суффиксы).
- Эффективное решение проблемы OOV (разбиение до символов).

Недостатки:

- Жадный выбор префикса при инференсе не всегда идеален.
- Дублирование токенов с префиксом ##.

Используется в моделях **BERT, DistilBERT, ELECTRA,** ...

Заключение

Токенизация подсловами — оптимальный баланс между токенизацией словами и буквами.

Критерий	**По словам**	**По буквам**	**Подслова**
Размер словаря	Огромный	Минимальный	Оптимальный
Проблема OOV	Критична	Отсутствует	Отсутствует
Длина послед-ти	Короткая	Очень длинная	Средняя
Семантика	Сохранена	Потеряна	Сохранена
Морфология	Не видна	Размыта	Выделена

- Подслова позволяют работать с языками с богатой морфологией (русский, турецкий).
- Модель понимает связь между *play, playing, player*, разделяя их на общие части.
- Модель может генерировать новые (не из обучения) формы слов.
- Все современные LLM используют именно этот подход.